

UNIT – 3
SYNCHRONIZATION AND DEADLOCKS

3.0 Synchronization & Deadlocks

3.1 Describe semaphores

3.2 Explain inter process communication

3.3 Define Deadlock

3.4 State the necessary conditions for arising deadlocks

3.5 State various techniques for deadlock prevention

3.6 Discuss Deadlock avoidance and detection

3.7 Describe the process of recovering from deadlock

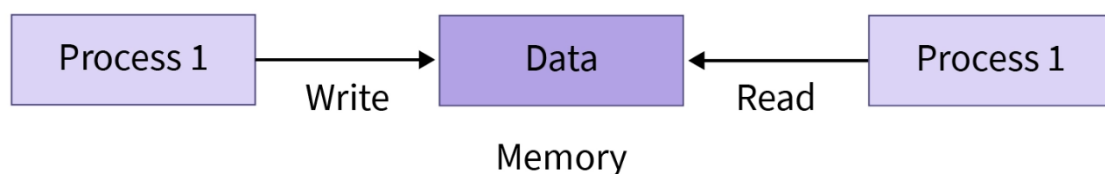
Synchronization

Synchronization is the process of controlling the execution of multiple processes or threads so that they can safely share resources like memory, files, or printers without causing conflicts. It ensures that only one process enters a critical section a part of the code where the shared resource is accessed at a time. Without proper synchronization, problems like race conditions can occur, where the output depends on the unpredictable order of execution.

Definition of Process Synchronization

Process Synchronization is a technique used in operating systems to control the execution order of processes or threads that share common resources like memory, files, or printers. Its main goal is to prevent data inconsistency and ensure that only one process accesses a shared resource at a time. Synchronization is especially important in multi-processing or multi-threading environments, where multiple processes may try to perform operations on the same resource simultaneously

Process synchronization in Operating System



Working:

Synchronization uses tools like semaphores, mutexes, or monitors to lock and unlock access to critical resources. When one process is using a shared resource, others are forced to wait until it's released.

Example:

In a banking application, if two users try to withdraw money from the same account at the same time, synchronization ensures that one transaction completes before the other begins to avoid wrong balance updates.

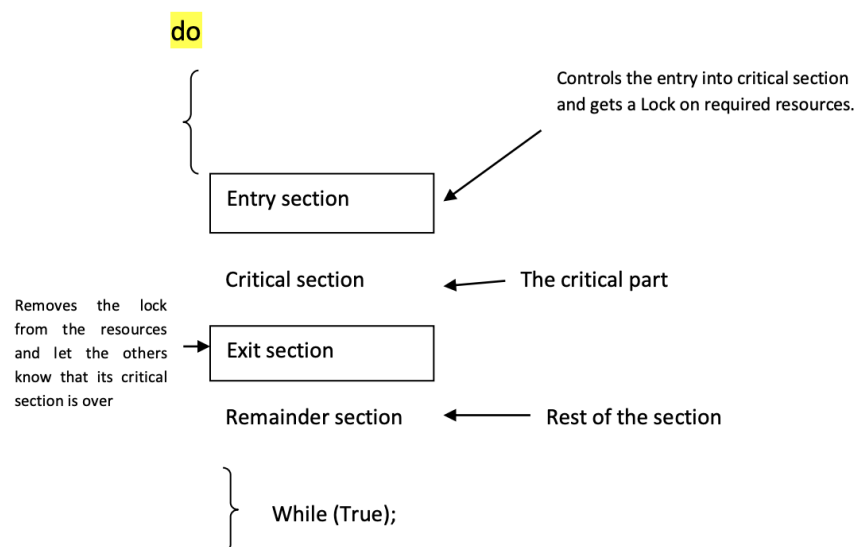
On the basis of synchronization, processes are categorized as one of the following two types:

- **Independent Process:** The execution of one process does not affect the execution of other processes.
- **Cooperative Process:** A process that can affect or be affected by other processes executing in the system.

Critical Section Problem in Operating System

The Critical Section Problem refers to a situation in an operating system where multiple processes or threads need to access shared resources (like variables, files, or memory) but only one process should access the critical section at a time to prevent errors.

The critical section is a part of the program where the shared resource is used. If two processes enter the critical section at the same time, it can cause data inconsistency, corruption, or crashes.



To solve this problem, three rules must be followed:

1. **Mutual Exclusion** – Only one process can be in the critical section at a time.
2. **Progress** – If no process is in the critical section, others should not be blocked unnecessarily.

3. **Bounded Waiting** – Each process must get a chance to enter the critical section after a limited number of turns.

Example:

Imagine two users trying to edit the same document online at the same time. If there's no control (synchronization), their changes might overwrite each other. The system uses a critical section mechanism to allow only one user to edit at a time to ensure accuracy.

Semaphore

Semaphores are integer variables that are used to solve the critical section problem by using two atomic operations, wait and signal that are used for process synchronization.

Semaphores are integer variables that can be modified using two atomic operations:

- wait() (also called P): decreases the value and may block the process if the value becomes negative.
- signal() (also called V): increases the value and may wake up a blocked process.

Example Scenario: Printer Access

Imagine two processes, P1 and P2, both trying to use a printer.

We define a binary semaphore $S = 1$ (1 means the printer is free).

```
Semaphore S = 1; // Initially printer is free

// Process P1
wait(S);        // S becomes 0 → P1 enters critical section
// Use printer
signal(S);       // S becomes 1 → Printer is released

// Process P2
wait(S);        // If S is 0, P2 will wait
// Use printer
signal(S);       // S becomes 1
```

How it works:

1. P1 calls wait(S) → $S = 1$ becomes $S = 0$ → allowed to use the printer.
2. While P1 is using the printer, P2 calls wait(S) → $S = 0$ → P2 is blocked.

3. P1 finishes and calls $\text{signal}(S) \rightarrow S = 1$.
4. P2 is unblocked, gets access to the printer.

Types of Semaphores:

1. Binary Semaphore (also called Mutex)

- It has only two values: 0 and 1.
- Used to allow or block access to a single resource.
- Works like a lock:
 - 1 means the resource is free,
 - 0 means the resource is occupied.
- Only one process can enter the critical section at a time.

Example: Only one person can use an ATM at a time — others must wait until it's free.

2. Counting Semaphore

- It can have any non-negative integer value.
- Used when multiple instances of a resource are available.
- The value represents how many processes can access the resource at the same time.

Example: A parking lot with 5 spaces $\rightarrow$ 5 cars can enter (semaphore = 5). Each car decreases the count by 1; when a car leaves, it increases by 1.

Advantages of Semaphore:

1. **Avoids Race Conditions**
Ensures that only one process accesses a shared resource at a time, preventing conflicts.
2. **Efficient Resource Management**
Helps manage access to limited resources like printers, memory, or CPU.
3. **Supports Multiprocessing**
Allows smooth coordination between multiple processes or threads.

Disadvantages of Semaphore:

1. **Complex to Implement**
Requires careful programming; mistakes can cause serious issues.
2. **Deadlock Risk**
Poor use of semaphores can still lead to deadlocks if processes wait forever.
3. **Busy Waiting (in some cases)**
A process might keep checking (polling) the semaphore instead of doing useful work.

Example of Semaphore:

For example, Suppose there are 4 processes P1, P2, P3, P4, and they all call wait operation on S(initialized with 4). If another process P5 wants the resource then it should wait until one of the four processes calls the signal function and the value of semaphore becomes positive.

Uses Of Semaphore:

Semaphores can be used for three purposes: -

- To ensure mutually exclusive execution of a critical section (as locks do).
- To control access to a shared pool of resources (using a counting semaphore). -
- To cause one thread to wait for a specific action to be signalled from another thread.

Inter Process Communication(IPC)

Interprocess communication is the mechanism provided by the operating system that allows processes to communicate with each other. This communication could involve a process letting another process know that some event has occurred or the transferring of data from one process to another.

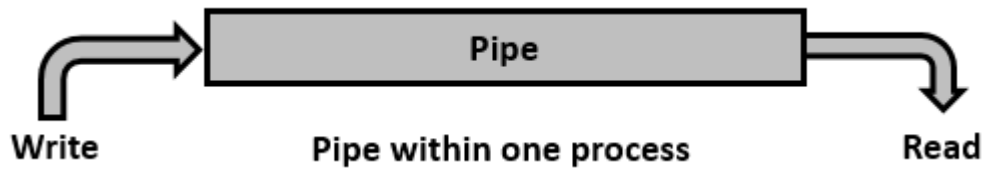


Categories of Interprocess Communication

- 1) Pipes
- 2) FIFO's
- 3) Shared Memory
- 4) Message Passing
- 5) Sockets

1. Pipes

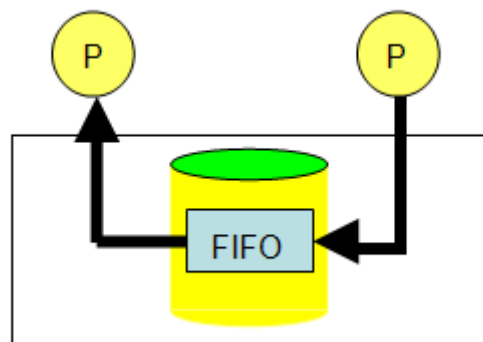
Pipes are used for unidirectional communication between two related processes, such as a parent and its child. Data flows in one direction only, from the writer process to the reader. Pipes are temporary and do not have a name in the file system. They are commonly used in programs where output of one process is input to another.



Example: A parent process creates a pipe to send a message to its child process during execution.

2. FIFOs (Named Pipes)

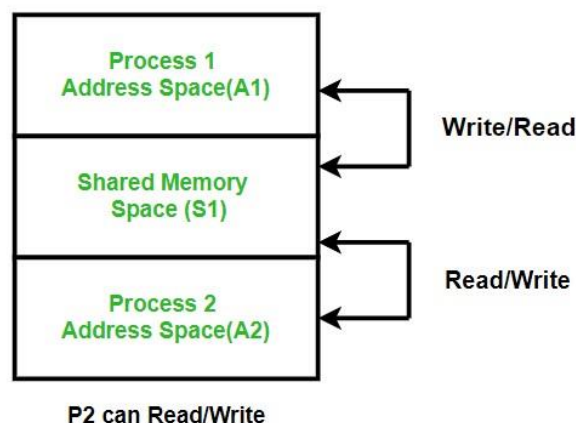
FIFOs are similar to pipes but are named and stored as special files in the file system. They allow communication between unrelated processes and support bidirectional communication. Unlike pipes, FIFOs exist even after the processes end until the file is removed. They are useful for command-line tools or inter-process messaging.



Example: Two independent applications use a named FIFO file to exchange data in a Linux system.

3. Shared Memory

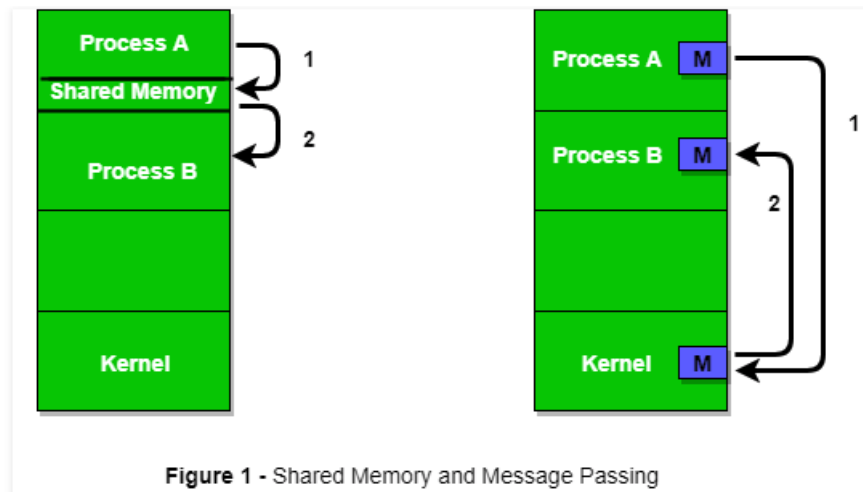
Shared memory allows multiple processes to access a common memory region, which makes it the fastest IPC method. Since the memory is shared, processes need synchronization tools like semaphores to avoid data conflicts. Shared memory is efficient for large amounts of data. It is commonly used in real-time systems.



Example: A producer process writes data into shared memory, and a consumer process reads it continuously.

4. Message Passing

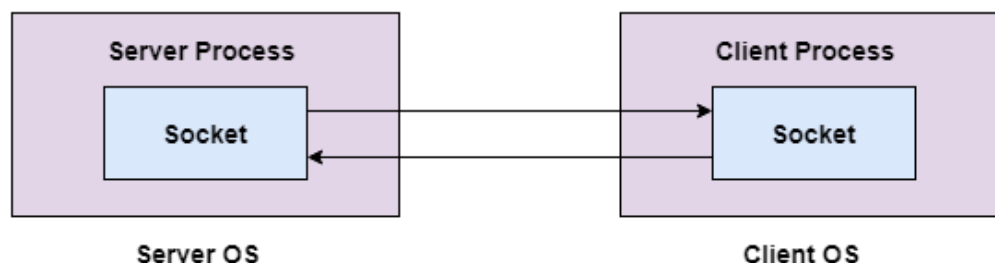
Message passing allows processes to send and receive messages through the OS without sharing memory. It is safe and structured, especially in distributed systems where processes run independently. Messages can be simple data or full objects. Though slower than shared memory, it is easier to manage and secure.



Example: An operating system uses message queues for sending job status between scheduling modules.

5. Sockets

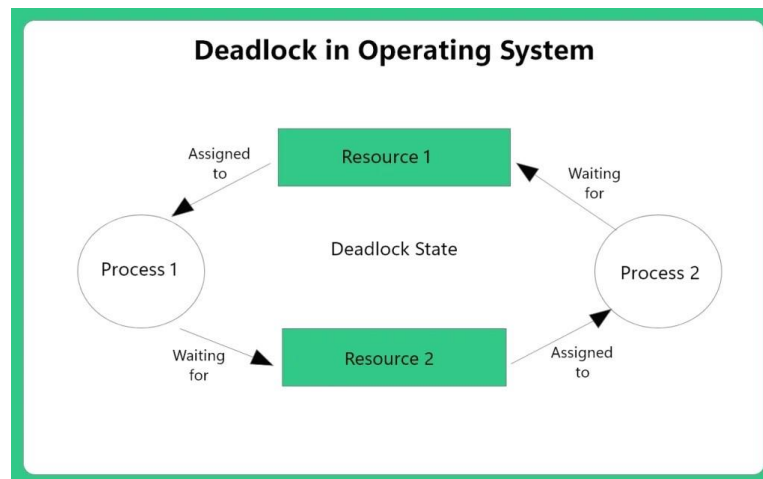
Sockets are used for communication between processes over a network, such as between a client and a server. They support bidirectional communication and use internet protocols like TCP/IP. Sockets allow processes on different machines to interact in real time. This is the foundation of most network-based applications.



Example: A web browser communicates with a web server using sockets over the internet to load a webpage.

Deadlock

Deadlock is a situation in an operating system where two or more processes are waiting for each other to release resources, and none of them can continue. Each process holds a resource and waits for another resource that is being held by another process. As a result, all the processes get stuck and cannot proceed. Deadlock leads to infinite waiting unless the system takes action to break it.



1. Mutual Exclusion

At least one resource must be held in a non-shareable mode, meaning only one process can use it at a time. If another process requests that resource, it must wait until it's released.

Example: A printer being used by one process cannot be used by another process at the same time.

2. Hold and Wait

A process is holding at least one resource and is waiting to acquire additional resources that are currently held by other processes.

Example: Process A holds a scanner and waits for a printer, while another process holds the printer.

3. No Preemption

Resources cannot be forcibly taken away from a process holding them; they can only be released voluntarily by the process after its task is complete.

Example: The system cannot take back a file lock from a process until the process releases it.

4. Circular Wait

A set of processes exists such that each process is waiting for a resource held by the next process in the chain, forming a circular loop.

Example:

Process A waits for B's resource,
B waits for C's resource,
C waits for A's resource → circular wait occurs.

Various Techniques For Deadlock Prevention

Deadlock prevention means designing a system in such a way that at least one of the four necessary conditions for deadlock can never occur. This helps avoid deadlocks before they happen.

Here are the main techniques used:

1. Mutual Exclusion Removal

Try to avoid exclusive access to resources wherever possible.
Make resources sharable, like read-only files that can be accessed by multiple processes. However, some resources (like printers) must be exclusive, so this method is not always practical.

Example: Allowing multiple processes to read the same file simultaneously.

2. Eliminating Hold and Wait

Make sure that a process requests all required resources at once, before it begins execution. If the resources are not available, the process must wait without holding any resources. This avoids holding one resource while waiting for others, but it may lead to resource wastage.

Example: A process asks for both a printer and a scanner at the same time before starting.

3. No Preemption

Allow preemption of resources — forcibly take a resource from a process and give it to another.

If a process holding some resources requests another unavailable resource, then its held resources are taken away, and it waits until all resources are available.

This can be complex to implement and may cause loss of data.

Example: Temporarily removing memory from one process to give it to another high-priority process.

4. Avoiding Circular Wait

Assign a fixed order (numbering) to all resources, and require that processes request resources in increasing order.

This prevents the formation of circular chains of waiting processes, thus breaking the circular wait condition.

Example: If Resource A is 1 and Resource B is 2, a process must request Resource A before Resource B.

Deadlock Avoidance and Detection

Deadlock Avoidance:

- In deadlock avoidance, the request for any resource will be granted if the resulting state of the system doesn't cause deadlock in the system.
- In order to avoid deadlocks, the process must tell OS, the maximum number of resources a process can request to complete its execution.
- Based on these criteria, the operating system decides which process sequence should be executed or waited so that no deadlock occurs in a system.
- Therefore, it is also known as deadlock avoidance algorithm or deadlock detection in the operating system.
- The Banker's algorithm is a resource allocation and deadlock avoidance algorithm developed by Edsger Dijkstra

Why it's called "Banker's Algorithm"?

It is named after the way a banker lends money:

A banker only gives out loans if, even in the worst case, the bank can still satisfy all customers' maximum demands. Similarly, the OS only allocates resources if it can still safely serve all processes.

Here we check

- $Need[i] \leq work$
- $Work = Work + Allocation(i)$
- $Finish[i] = true$
- Go to step 2 to check the status of resource availability for the next process.
- If $Finish[i] == true$; it means that the system is safe for all processes.

Let us consider the following example for understanding the banker's algorithm:

Example: Consider a system that contains five processes P1, P2, P3, P4, P5 and the three resource types A, B and C. Following are the resources types: A has 10, B has 5 and the resource type C has 7 instances.

Given:

- **Processes:** P1, P2, P3, P4, P5
- **Resources:** A = 10, B = 5, C = 7

Step 1: Data Given

Process	Allocation (A B C)	Max (A B C)
P1	0 1 0	7 5 3
P2	2 0 0	3 2 2
P3	3 0 2	9 0 2
P4	2 1 1	2 2 2
P5	0 0 2	4 3 3

Now calculate the total allocated resources to get the Available:

- **Total Allocation:**
 - $A = 0 + 2 + 3 + 2 + 0 = 7$
 - $B = 1 + 0 + 0 + 1 + 0 = 2$
 - $C = 0 + 0 + 2 + 1 + 2 = 5$
- **Available Resources = Total - Allocated**
 - $A = 10 - 7 = 3$
 - $B = 5 - 2 = 3$
 - $C = 7 - 5 = 2$

So,

Available = [3 3 2]

Step 2: Calculate the Need Matrix (Need = Max – Allocation)

Process	Need (A B C)
P1	$7-0 \quad 5-1 \quad 3-0 = [7 \ 4 \ 3]$
P2	$3-2 \quad 2-0 \quad 2-0 = [1 \ 2 \ 2]$
P3	$9-3 \quad 0-0 \quad 2-2 = [6 \ 0 \ 0]$
P4	$2-2 \quad 2-1 \quad 2-1 = [0 \ 1 \ 1]$
P5	$4-0 \quad 3-0 \quad 3-2 = [4 \ 3 \ 1]$

Step 3: Apply Banker's Algorithm

We now try to find a safe sequence using the Need and Available:

Current Available = [3 3 2]

Check each process:

- $P1 \rightarrow \text{Need } [7\ 4\ 3] > \text{Available} \rightarrow \times \text{ False}$
- $P2 \rightarrow \text{Need } [1\ 2\ 2] \leq \text{Available} \rightarrow \checkmark \text{ True}$
 - P2 can complete.
 - $\text{New Available} = \text{Available} + \text{Allocation of P2}$
 $= [3+2, 3+0, 2+0] = [5\ 3\ 2]$

☒ Safe sequence so far: **P2**

Available = [5 3 2]

- $P1 \rightarrow \text{Need } [7\ 4\ 3] > \text{Available} \rightarrow \times \text{ False}$
- $P3 \rightarrow \text{Need } [6\ 0\ 0] > [5\ 3\ 2] \rightarrow \times \text{ False}$
- $P4 \rightarrow \text{Need } [0\ 1\ 1] \leq \text{Available} \rightarrow \checkmark \text{ True}$
 - $\text{New Available} = [5+2, 3+1, 2+1] = [7\ 4\ 3]$

☒ Safe sequence: **P2 $\rightarrow$ P4**

Available = [7 4 3]

- $P1 \rightarrow \text{Need } [7\ 4\ 3] \leq \text{Available} \rightarrow \checkmark \text{ True}$
 - $\text{New Available} = [7+0, 4+1, 3+0] = [7\ 5\ 3]$

☒ Safe sequence: **P2 $\rightarrow$ P4 $\rightarrow$ P1**

Available = [7 5 3]

- $P3 \rightarrow \text{Need } [6\ 0\ 0] \leq \text{Available} \rightarrow \checkmark \text{ True}$
 - $\text{New Available} = [7+3, 5+0, 3+2] = [10\ 5\ 5]$

☒ Safe sequence: **P2 $\rightarrow$ P4 $\rightarrow$ P1 $\rightarrow$ P3**

Available = [10 5 5]

- $P5 \rightarrow \text{Need } [4\ 3\ 1] \leq \text{Available} \rightarrow \checkmark \text{ True}$
 - $\text{New Available} = [10+0, 5+0, 5+2] = [10\ 5\ 7]$

☒ Final Safe sequence:

P2 $\rightarrow$ P4 $\rightarrow$ P1 $\rightarrow$ P3 $\rightarrow$ P5

Conclusion:

☒ The system is in a **SAFE STATE**

☒ Safe Sequence: **P2 $\rightarrow$ P4 $\rightarrow$ P1 $\rightarrow$ P3 $\rightarrow$ P5**

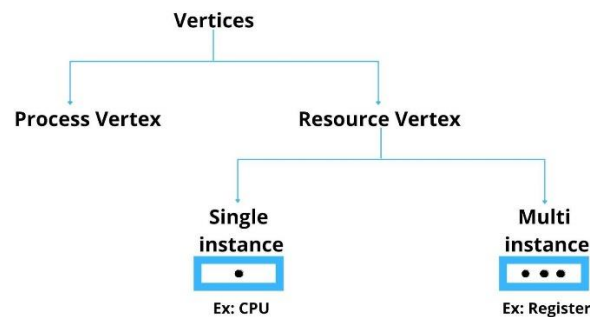
DEADLOCK DETECTION:

Deadlock avoidance is a technique used to prevent a system from entering a deadlock state. It works by checking before allocating a resource whether doing so might lead to deadlock. If the system is safe, the resource is allocated; otherwise, the process must wait.

Resource Allocation Graph (RAG)

A Resource Allocation Graph (RAG) is a diagram used in operating systems to represent the relationship between processes and resources in a system. It helps in detecting and understanding deadlocks by showing which processes are holding or waiting for resources. In this graph, processes are represented as circles (called process vertices), and resources are shown as rectangles (resource vertices).

A vertex is a point or node in a graph that represents either a process or a resource.



Process vertex

A Process vertex is a node in a Resource Allocation Graph that represents an active process in the system. It may request resources (shown by edges from the process to resources) or hold resources (shown by edges from resources to the process). These vertices help visualize process-resource interactions, especially in deadlock detection.

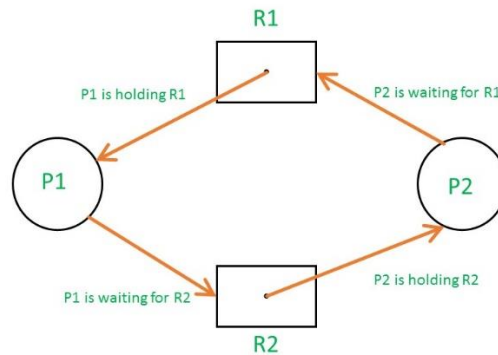
Resource vertex

A Resource vertex is a node in a Resource Allocation Graph that represents a system resource. It can have one or more instances and is connected to processes either by request edges (from process to resource) or assignment edges (from resource to process). Resource vertices help illustrate how resources are allocated or requested by processes.

Single instance type resource

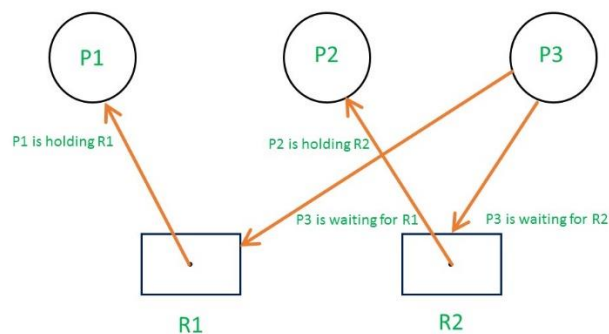
A Single instance type resource is a resource that has only one unit available in the system. At any given time, only one process can hold or use this resource. If another process requests it while it's in use, that process must wait until it is released.

Example 1 (Single Instances RAG)



SINGLE INSTANCE RESOURCE TYPE WITH DEADLOCK

If there is a cycle in the Resource Allocation Graph and each resource in the cycle provides only one instance, then the processes will be in deadlock. For example, if process P1 holds resource R1, process P2 holds resource R2 and process P1 is waiting for R2 and process P2 is waiting for R1, then process P1 and process P2 will be in deadlock.



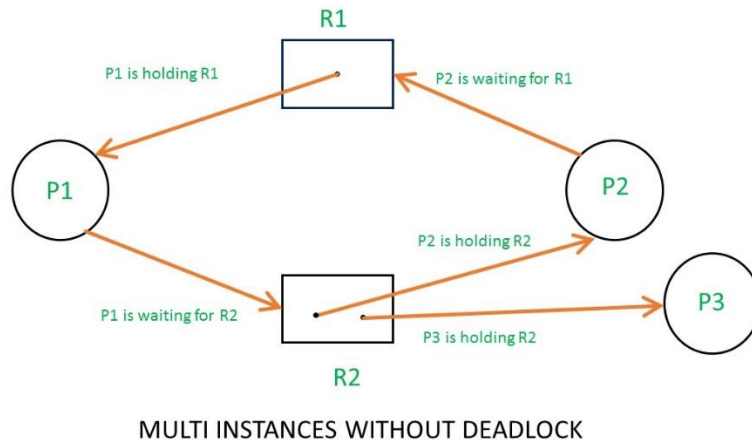
SINGLE INSTANCE RESOURCE TYPE WITHOUT DEADLOCK

Here's another example, that shows Processes P1 and P2 acquiring resources R1 and R2 while process P3 is waiting to acquire both resources. In this example, there is no deadlock because there is no circular dependency. So cycle in single-instance resource type is the sufficient condition for deadlock.

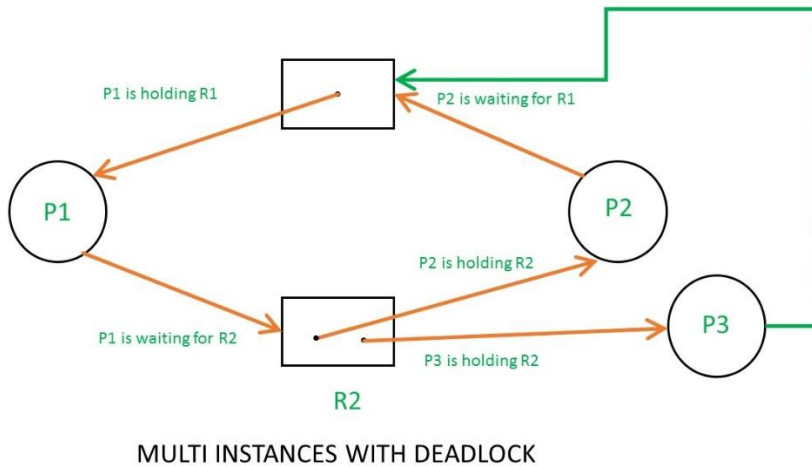
Multi-resource instance type resource

A Multi - resource instance type resource is a resource that has multiple identical units available in the system. Several processes can hold different instances of this resource at the same time. Allocation and request are managed based on the number of available instances.

Example 2 (Multi-instances RAG)



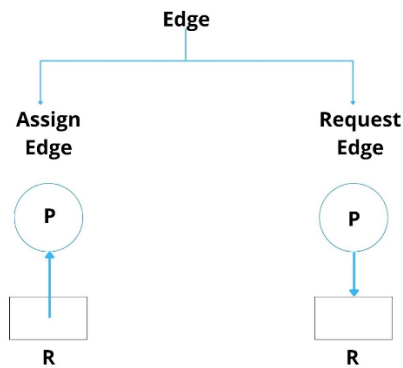
From the above example, it is not possible to say the RAG is in a safe state or in an unsafe state. Because, with two instances, it is going to release the resource after completion



Therefore, every cycle in a multi-instance resource type graph is not a deadlock, if there has to be a deadlock, there has to be a cycle. So, in case of RAG with multiinstance resource type, the cycle is a necessary condition for deadlock, it is rectified easily with multiple instances.

Types of Edges in Resource Allocation Graph (RAG)

Edges: A line or arrow connecting two vertices (nodes) in a graph, showing the relationship between a process and a resource.



In a Resource Allocation Graph (RAG), there are two main types of edges that represent the relationship between processes and resources. The first is the Request Edge, which is a directed arrow from a process to a resource. This indicates that the process is requesting or waiting for that resource.

The second is the Assignment Edge, which is a directed arrow from a resource to a process, showing that the resource has been allocated to the process. These edges help the operating system understand which resources are in use and which processes are waiting, and are essential for analyzing and detecting deadlocks in the system.

Recovering From Deadlock

When a system enters a deadlock state, it means that a group of processes are stuck, each waiting for resources held by others, and none can continue. Deadlock recovery refers to the methods used to break this situation and bring the system back to normal operation. There are several ways to recover from a deadlock:

1. Process Termination

- Terminate one or more processes involved in the deadlock.
- You can either kill all deadlocked processes (fast but harsh) or terminate them one by one until the deadlock is broken.
- This frees the resources and allows remaining processes to continue.

Example: Terminate Process P3 to free Resource R2.

Types of Process Termination in Deadlock Recovery

1. Terminate All Deadlocked Processes

- In this method, all processes involved in the deadlock are forcefully terminated.
- It is the simplest and fastest way to recover from a deadlock.
- However, it can lead to loss of work and data inconsistency, especially if the processes were running for a long time.

Example: If P1, P2, and P3 are in a deadlock, the system kills all three

2. Terminate One Process at a Time

- Here, the system terminates one process at a time from the deadlocked group.
- After each termination, it checks if the deadlock is resolved.
- This approach is safer, but it takes more time to break the deadlock.

Example: Kill P3 → Check if deadlock resolved → If not, kill P2, and so on.

2. Resource Preemption

- Temporarily take resources away from one process and give them to another.
- The process whose resource is taken must wait and may be resumed later.
- Needs careful handling to avoid data loss or inconsistency.

Example: Take Resource R1 from P2 and give to P1.

(a). Selecting a victim: We must determine which resources and which processes are to be preempted and also the order to minimize the cost.

(b). Rollback: We must determine what should be done with the process from which resources are preempted. One simple idea is total rollback. That means abort the process and restart it.

(c). Starvation: In a system, it may happen that same process is always picked as a victim. As a result, that process will never complete its designated task. This situation is called Starvation and must be avoided. One solution is that a process must be picked as a victim only a finite number of times.

IMPORTANT QUESTIONS:

Shorts:

- 1) Define process synchronization
- 2) What is semaphore? Give an example?
- 3) Write the uses of semaphore?
- 4) What is critical section problem & solution?
- 5) Define deadlock with example?
- 6) List the overall deadlock handling methods?
- 7) Write about necessary conditions for deadlock?
- 8) What is starvation problem?
- 9) List the two ways that we can break the second condition to prevent deadlock?

Essays:

1. Explain about semaphores and its operation?
2. Explain about Inter Process Communication?
3. Explain about deadlock prevention methods?
4. Explain about deadlock avoidance methods?
5. Explain deadlock recovery process?
6. Explain about deadlock detection?